Int J Softw Tools Technol Transfer (2014) 16:123–125
DOI 10.1007/s10009-013-0297-7

INTRODUCTION

Developments in automated verification techniques

Cormac Flanagan · Barbara König

Published online: 20 December 2013
© Springer-Verlag Berlin Heidelberg 2013

Abstract Tools that implement automated verification techniques can be used to fruitfully analyze and validate complex software systems. Developing such tools is an active research area that has produced several promising techniques in the last decade: however, many challenges lie ahead. We briefly review the research area and summarize four papers selected from the Eighteenth International Conference on Tools and Algorithms for the Construction and Analysis of Systems (TACAS 2012).

Keywords Verification · Analysis tools · Model-checking

1 Background

Software errors can cause severe problems for our society. They can be as spectacular as bugs in embedded systems, for instance in aerospace or automotive software, potentially causing loss of life. Or they can appear in applications that carry less risks, such as text-processing software or software for mobile devices, where they cause less danger, but are still annoying for the user, reducing productivity and potentially compromising security.

Every engineering discipline must have methods to validate and verify its products, ensuring high standards of quality. For computer science, this challenge is aggravated by the rapid development of the discipline over the last few decades, with software evolving from programs running on single computers to highly concurrent and distributed systems.

In industry, software testing is widely established as an important means of validation. On the other hand, citing Edsger W. Dijkstra: "Program testing can be a very effective way to show the presence of bugs, but is hopelessly inadequate for showing their absence." Hence, other techniques are needed and are now starting to find their foothold in industrial applications: theorem proving, which requires the interaction with a suitably trained user, and automated verification techniques, i.e., push-button methods that take as input code (or a model of the system) and a specification (given, for instance, by a logical formula) and try to check that the system satisfies the specification.

However, such push-button methods suffer from well-known theoretical limitations: it follows from Rice's theorem [13] that it is undecidable for every (non-trivial) property of functions whether the function computed by a program has this specific property. Despite this negative result, known now for a long time, many fruitful methods for automated verification have emerged. First, there are model-checking methods that can efficiently deal with huge finite state spaces. Second, certain classes of infinite-state systems do have decidable verification problems. And third, successful abstraction techniques (such as predicate abstraction and counterexample-guided abstraction refinement) have emerged that offer approximate answers.

Naturally, in practical applications, these techniques need not be applied separately, but can be fruitfully combined. All such methods and their integration are in the scope of the Conference on Tools and Algorithms for the Construction and Analysis of Systems (TACAS).

Selected papers from TACAS 2012 (Eighteenth International Conference on Tools and Algorithms for the Construction and Analysis of Systems).

C. Flanagan
University of California, Santa Cruz, USA
e-mail: cormac@ucsc.edu

B. König (✉)
University of Duisburg-Essen, Duisburg, Germany
e-mail: barbara_koenig@uni-due.de

A very important aspect is tool development, since no method can be effectively tested and applied in practice without tool support. To encourage tool development and to evaluate tools, TACAS 2012 has hosted the competition on software verification. This was the first such competition event with the goal to evaluate the technology and compare state-of-the-art software verifiers with respect to effectiveness and efficiency. The competition was performed and presented by the TACAS Competition Chair Dirk Beyer.

2 Developments

Despite the problems mentioned above and the fact that many problems still lie ahead, there have been several encouraging developments in the last few years. Here is a non-exhaustive list.

- Abstraction introduces over-approximation, which in turn leads to spurious error messages and counterexamples. Abstraction refinement techniques, especially *counter-example-guided abstraction refinement* [10], modify the abstraction such that a given spurious counterexample disappears. Iterating this method leads to successful verification in many cases.
- *Bounded model checking* [5] harnesses the strength of *SAT-solvers* to automatically check that no error appears up to a given number of steps. This is done by generating a propositional formula that is checked for satisfiability by an optimized, special-purpose SAT-solver.
- Quantitative information can be handled by *SMT-solvers* [1], which address satisfiability problems for logical formulas with background theories.
- In general, one can observe a trend in verification where models are enriched with *quantitative* information, which allows more than simply "yes or no" queries. Instead, verification yields numerical information such as probabilities, time, weights or distances.
- A further research direction relies on decidability results for *infinite-state systems* [9]. For instance, using decidability results for pushdown systems [4,8], one can verify recursive function calls. This approach is known as procedural model checking (see also [3]). By bounding certain parameters, for instance the number of context switches [12], such techniques can be extended to multi-threaded programs.
- Dynamically evolving graph-like structures, such as the program heap or ad-hoc networks, can be analyzed using techniques such as separation logic [7], 3-valued shape analysis [14] or methods based on graph rewriting and formal language theory.
- Automated verification techniques can handle increasingly complex case studies and tool support is becoming more sophisticated. This is also witnessed by the software model-checking competition that accompanied TACAS.

Despite all this progress, many challenges remain. One important issue is *scalability*, i.e., can the techniques be applied to large programs with millions of lines of code? To solve this problem in a satisfying way, more results on compositional and modular verification techniques are needed. Furthermore, as mentioned earlier, the development of methods has to keep up with the complexity of systems that is increasing quickly. Problems are caused by features such as weak memory models, complex data structures, distributed and concurrent programs, protocols depending on probabilities and time and security-critical applications.

3 Selected papers

The papers selected for this special issue cover several of the diverse topics mentioned above. They were invited as the best papers related to tool implementation and case studies that were submitted to the conference. All four of them make important steps in addressing and solving the current challenges in program verification and analysis.

Bounded Phase Analysis of Message-Passing Pro-grams [2] by Bouajjani and Emmi describes a verification technique for infinite-state systems. It shows how to analyze asynchronous message-passing programs with ordered message queues, where the bounding parameter limits the number of process communication cycles. The analysis scales independently of queue content.

Similarly, *Pushdown Model Checking for Malware Detection* [15] by Song and Touili is based on techniques for infinite-state systems. It makes an original contribution by modelling programs including the stack by pushdown systems and using a model-checking procedure for temporal logics to detect malware. The authors implemented their technique in a tool and applied it to detect several viruses.

The paper *A Bit Too Precise? Verification of Quantized Digital Filters* [6] by Cox et al. is concerned with quantitative system verification. It considers fixed point digital filters which are a simple yet ubiquitous component of digital processing and control systems. The authors study two classes of verification techniques: bit-precise analysis and real-valued error approximations, which they evaluate empirically. They suggest efficient approaches using SMT-solvers and hardware model checkers.

Finally, *Closed-loop Verification of Medical Devices with Model Abstraction and Refinement* [11] by Jiang et al. is a case study paper that makes an important contribu-

tion to the design and verification of medical devices. The authors model a dual chamber implantable pacemaker with timed automata and verify it using the tool UPPAAL.

4 Thanks

We would like to conclude by thanking the programme committee of TACAS 2012 and the reviewers of the present special section for their detailed evaluations and insightful comments.

References

1. Armando, A., Mantovani, J., Platania, L.: Bounded model checking of software using SMT solvers instead of SAT solvers. Int. J. Softw. Tools Technol. Transf. **11**(1), 69–83 (2009)
2. Bouajjani, A., Emmi, M.: Bounded phase analysis of message-passing programs. Int. J. Softw. Tools Technol. Transf. doi:10.1007/s10009-013-0276-z
3. Burkart, O., Steffen, B.: Model checking for context-free processes. In Proceedings of CONCUR '92, pp. 123–137. Springer, LNCS 630 (1992)
4. Burkart, O., Steffen, B.: Pushdown processes: parallel composition and model checking. In: Proceedings of CONCUR '94, pp. 98–113. Springer, LNCS 836 (1994)
5. Clarke, E., Biere, A., Raimi, R., Zhu, Y.: Bounded model checking using satisfiability solving. Formal Methods Syst. Des. **19**(1), 7–34 (2001)
6. Cox, A., Sankaranarayanan, S., Evan Chang, B.-Y.: A bit too precise? Verification of quantized digital filters. Int. J. Softw. Tools Technol. Transf. doi:10.1007/s10009-013-0279-9
7. Distefano, D., O'Hearn, P.W., Yang, H.: A local shape analysis based on separation logic. In: Proceedings of TACAS '06, pp. 287–302. Springer, LNCS 3920 (2006)
8. Esparza, J., Hansel, D., Rossmanith, P., Schwoon, S.: Efficient algorithms for model checking pushdown systems. In: Proceedings of CAV '00, pp. 232–247. Springer, LNCS 1855 (2000)
9. Finkel, A., Schnoebelen, P.: Well-structured transition systems everywhere! Theor. Comput. Sci. **256**(1–2), 63–92 (2001)
10. Henzinger, T.A., Jhala, R., Majumdar, R., McMillan, K.L.: Abstractions from proofs. In: Proceedings of POPL '04, pp. 232–244. ACM (2004)
11. Jiang, Z., Pajic, M., Alur, R., Mangharam, R.: Closed-loop verification of medical devices with model abstraction and refinement. Int. J. Softw. Tools Technol. Transf. doi:10.1007/s10009-013-0289-7
12. Qadeer, S., Rehof, J.: Context-bounded model checking of concurrent software. In Proceedings of TACAS '05, pp. 93–107. Springer, LNCS 3440 (2005)
13. Rice, H.G.: Classes of recursively enumerable sets and their decision problems. Trans. Am. Math. Soc. **74** (1953)
14. Sagiv, M., Reps, T., Wilhelm, R.: Parametric shape analysis via 3-valued logic. TOPLAS (ACM Trans. Program. Lang. Syst.) **24**(3), 217–298 (2002)
15. Song, F., Touili, T.: Pushdown model-checking for malware detection. Int. J. Softw. Tools Technol. Transf. doi:10.1007/s10009-013-0290-1